In [1]:
```
# ============================================================
# MEMBER 2 — DATA PREPROCESSING
# STEPS 1–21
# ============================================================

# Run this member's file first. It loads the original dataset,
# preprocesses it, and saves output/preprocessing_results/hotel_bookings_cleaned


# ============================================================
# HOTEL BOOKING CANCELLATION PREDICTION & ANALYSIS
# COMPLETE PROJECT — STEPS 1–90
# MEMBER 2: DATA PREPROCESSING — STEPS 1–21
# MEMBER 3: EDA & STATISTICAL ANALYSIS — STEPS 22–56
# MEMBER 4: VISUALIZATION & INTERPRETATION — STEPS 57–70
# MEMBER 5: MACHINE LEARNING & FINAL REPORT — STEPS 71–90
# ============================================================

# ============================================================
# STEP 1 — IMPORT LIBRARIES
# ============================================================

import pandas as pd
import numpy as np
import os
import warnings
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")

print("=" * 70)
print("HOTEL BOOKING CANCELLATION PREDICTION & ANALYSIS")
print("=" * 70)


print("Machine learning libraries imported successfully!")
print("Libraries imported successfully!")
```

```
======================================================================
HOTEL BOOKING CANCELLATION PREDICTION & ANALYSIS
======================================================================
Machine learning libraries imported successfully!
Libraries imported successfully!
```

In [2]:
```
# ============================================================
# STEP 2 — IMPORT MACHINE LEARNING LIBRARIES
# ============================================================

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    average_precision_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    RocCurveDisplay,
    PrecisionRecallDisplay
)
```

In [3]:
```
# ============================================================
# STEP 3 — SET DATASET PATH
# ============================================================

INPUT_FILE = (r"C:\Users\admin\OneDrive\Desktop\Hotel Booking Cancellation Predi
OUTPUT_FOLDER = "output"

PREPROCESSING_FOLDER = os.path.join(
    OUTPUT_FOLDER, "preprocessing_results"
)

EDA_FOLDER = os.path.join(
    OUTPUT_FOLDER, "eda_results"
)

VISUALIZATION_FOLDER = os.path.join(
    OUTPUT_FOLDER, "visualization_results"
)

ML_FOLDER = os.path.join(
    OUTPUT_FOLDER, "machine_learning_results"
)

for folder in [
    OUTPUT_FOLDER,
    PREPROCESSING_FOLDER,
    EDA_FOLDER,
    VISUALIZATION_FOLDER,
    ML_FOLDER
]:
    os.makedirs(folder, exist_ok=True)

print("Dataset path:", INPUT_FILE)
print("Output folders created successfully!")
```

```
Dataset path: C:\Users\admin\OneDrive\Desktop\Hotel Booking Cancellation Predicti
on & Analysis\hotel_bookings.csv
Output folders created successfully!
```

In [4]:
```
# ============================================================
# STEP 4 — LOAD ORIGINAL DATASET
# ============================================================
```

```
print("\n" + "=" * 70)
print("STEP 4 — LOAD ORIGINAL DATASET")
print("=" * 70)

if not os.path.exists(INPUT_FILE):
    raise FileNotFoundError(
        "Dataset not found. Please update INPUT_FILE "
        "with the correct hotel_bookings.csv path."
    )

df = pd.read_csv(INPUT_FILE)

print("Dataset loaded successfully!")
print("Rows    :", df.shape[0])
print("Columns :", df.shape[1])

print("\nFirst five records:")
print(df.head())
```

```
======================================================================
STEP 4 — LOAD ORIGINAL DATASET
======================================================================
Dataset loaded successfully!
Rows    : 119390
Columns : 33

First five records:
                        hotel  is_canceled  lead_time  arrival_date_year  \
0  Resort Hotel - Chandigarh            0        342               2024
1      Resort Hotel - Mumbai            0        737               2024
2       Resort Hotel - Delhi            0          7               2024
3     Resort Hotel - Kolkata            0         13               2024
4     Resort Hotel - Lucknow            0         14               2024

  arrival_date_month  arrival_date_week_number  arrival_date_day_of_month  \
0               July                        30                         27
1              April                        17                         28
2          September                        37                         10
3             August                        33                         14
4          September                        37                         14

   stays_in_weekend_nights  stays_in_week_nights  adults  ...  agent  company  \
0                        0                     0       2  ...    NaN      NaN
1                        0                     0       2  ...    NaN      NaN
2                        0                     1       1  ...    NaN      NaN
3                        0                     1       1  ...  304.0      NaN
4                        0                     2       2  ...  240.0      NaN

  days_in_waiting_list customer_type   adr required_car_parking_spaces  \
0                    0     Transient   0.0                           0
1                    0     Transient   0.0                           0
2                    0     Transient  75.0                           0
3                    0     Transient  75.0                           0
4                    0     Transient  98.0                           0

   total_of_special_requests  reservation_status  \
0                          0           Check-Out
1                          0           Check-Out
2                          0           Check-Out
3                          0           Check-Out
4                          1           Check-Out

         reservation_status_date        city
0  2024-07-27 22:16:40.916332324  Chandigarh
1  2024-04-28 21:56:21.507509066      Mumbai
2  2024-09-10 03:46:25.734029096       Delhi
3  2024-08-14 18:07:10.049669568     Kolkata
4  2024-09-14 14:27:32.473846000     Lucknow

[5 rows x 33 columns]
```

In [5]:

```
# ============================================================
# STEP 5 — BASIC DATASET INFORMATION
# ============================================================

print("\n" + "=" * 70)
print("STEP 5 — BASIC DATASET INFORMATION")
print("=" * 70)
```

```
print("\nDataset Shape:")
print(df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes)

print("\nDataset Information:")
df.info()
```

```
======================================================================
STEP 5 — BASIC DATASET INFORMATION
======================================================================

Dataset Shape:
(119390, 33)

Column Names:
['hotel', 'is_canceled', 'lead_time', 'arrival_date_year', 'arrival_date_month',
'arrival_date_week_number', 'arrival_date_day_of_month', 'stays_in_weekend_night
s', 'stays_in_week_nights', 'adults', 'children', 'babies', 'meal', 'country', 'm
arket_segment', 'distribution_channel', 'is_repeated_guest', 'previous_cancellati
ons', 'previous_bookings_not_canceled', 'reserved_room_type', 'assigned_room_typ
e', 'booking_changes', 'deposit_type', 'agent', 'company', 'days_in_waiting_lis
t', 'customer_type', 'adr', 'required_car_parking_spaces', 'total_of_special_requ
ests', 'reservation_status', 'reservation_status_date', 'city']

Data Types:
hotel                             str
is_canceled                     int64
lead_time                       int64
arrival_date_year               int64
arrival_date_month                str
arrival_date_week_number        int64
arrival_date_day_of_month       int64
stays_in_weekend_nights         int64
stays_in_week_nights            int64
adults                          int64
children                      float64
babies                          int64
meal                              str
country                           str
market_segment                    str
distribution_channel              str
is_repeated_guest               int64
previous_cancellations          int64
previous_bookings_not_canceled  int64
reserved_room_type                str
assigned_room_type                str
booking_changes                 int64
deposit_type                      str
agent                         float64
company                       float64
days_in_waiting_list            int64
customer_type                     str
adr                           float64
required_car_parking_spaces     int64
total_of_special_requests       int64
reservation_status                str
reservation_status_date           str
city                              str
dtype: object

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 119390 entries, 0 to 119389
Data columns (total 33 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   hotel                           119390 non-null  str
```

```
 1   is_canceled                     119390 non-null  int64
 2   lead_time                       119390 non-null  int64
 3   arrival_date_year               119390 non-null  int64
 4   arrival_date_month              119390 non-null  str
 5   arrival_date_week_number        119390 non-null  int64
 6   arrival_date_day_of_month       119390 non-null  int64
 7   stays_in_weekend_nights         119390 non-null  int64
 8   stays_in_week_nights            119390 non-null  int64
 9   adults                          119390 non-null  int64
 10  children                        119386 non-null  float64
 11  babies                          119390 non-null  int64
 12  meal                            119390 non-null  str
 13  country                         118902 non-null  str
 14  market_segment                  119390 non-null  str
 15  distribution_channel            119390 non-null  str
 16  is_repeated_guest               119390 non-null  int64
 17  previous_cancellations          119390 non-null  int64
 18  previous_bookings_not_canceled  119390 non-null  int64
 19  reserved_room_type              119390 non-null  str
 20  assigned_room_type              119390 non-null  str
 21  booking_changes                 119390 non-null  int64
 22  deposit_type                    119390 non-null  str
 23  agent                           103050 non-null  float64
 24  company                         6797 non-null    float64
 25  days_in_waiting_list            119390 non-null  int64
 26  customer_type                   119390 non-null  str
 27  adr                             119390 non-null  float64
 28  required_car_parking_spaces     119390 non-null  int64
 29  total_of_special_requests       119390 non-null  int64
 30  reservation_status              119390 non-null  str
 31  reservation_status_date         119390 non-null  str
 32  city                            119390 non-null  str
dtypes: float64(4), int64(16), str(13)
memory usage: 30.1 MB
```

In [6]:

```
# ============================================================
# STEP 6 — CHECK MISSING VALUES
# ============================================================

print("\n" + "=" * 70)
print("STEP 6 — MISSING VALUE ANALYSIS")
print("=" * 70)

missing_count = df.isnull().sum()
missing_percentage = (missing_count / len(df)) * 100

missing_analysis = pd.DataFrame({
    "Missing_Count": missing_count,
    "Missing_Percentage": missing_percentage.round(2)
})

missing_analysis = missing_analysis.sort_values(
    "Missing_Count",
    ascending=False
)

print(missing_analysis)

missing_analysis.to_csv(
    os.path.join(
```

```
        PREPROCESSING_FOLDER,
        "step_06_missing_value_analysis.csv"
    )
)
```

```
======================================================================
STEP 6 — MISSING VALUE ANALYSIS
======================================================================
                                Missing_Count  Missing_Percentage
company                                112593               94.31
agent                                  16340               13.69
country                                  488                0.41
children                                   4                0.00
hotel                                      0                0.00
arrival_date_week_number                   0                0.00
arrival_date_day_of_month                  0                0.00
is_canceled                                0                0.00
lead_time                                  0                0.00
stays_in_week_nights                       0                0.00
stays_in_weekend_nights                    0                0.00
babies                                     0                0.00
adults                                     0                0.00
meal                                       0                0.00
market_segment                             0                0.00
arrival_date_year                          0                0.00
arrival_date_month                         0                0.00
is_repeated_guest                          0                0.00
distribution_channel                       0                0.00
previous_cancellations                     0                0.00
previous_bookings_not_canceled             0                0.00
assigned_room_type                         0                0.00
reserved_room_type                         0                0.00
deposit_type                               0                0.00
booking_changes                            0                0.00
days_in_waiting_list                       0                0.00
customer_type                              0                0.00
adr                                        0                0.00
required_car_parking_spaces                0                0.00
total_of_special_requests                  0                0.00
reservation_status                         0                0.00
reservation_status_date                    0                0.00
city                                       0                0.00
```

In [7]:
```
# ============================================================
# STEP 7 — CHECK DUPLICATE RECORDS
# ============================================================

print("\n" + "=" * 70)
print("STEP 7 — DUPLICATE ANALYSIS")
print("=" * 70)

duplicate_count = df.duplicated().sum()

print("Duplicate records:", duplicate_count)
```

```
======================================================================
STEP 7 — DUPLICATE ANALYSIS
======================================================================
Duplicate records: 0
```

In [8]:
```
# ============================================================
# STEP 8 — REMOVE DUPLICATE RECORDS
# ============================================================

print("\n" + "=" * 70)
print("STEP 8 — REMOVE DUPLICATES")
print("=" * 70)

df = df.drop_duplicates().copy()

print("Duplicates removed.")
print("Updated dataset shape:", df.shape)
```

```
======================================================================
STEP 8 — REMOVE DUPLICATES
======================================================================
Duplicates removed.
Updated dataset shape: (119390, 33)
```

In [9]:
```
# ============================================================
# STEP 9 — IDENTIFY NUMERICAL FEATURES
# ============================================================

print("\n" + "=" * 70)
print("STEP 9 — NUMERICAL FEATURES")
print("=" * 70)

numeric_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

print("Numerical columns:")
print(numeric_columns)

print("Number of numerical features:", len(numeric_columns))
```

```
======================================================================
STEP 9 — NUMERICAL FEATURES
======================================================================
Numerical columns:
['is_canceled', 'lead_time', 'arrival_date_year', 'arrival_date_week_number', 'ar
rival_date_day_of_month', 'stays_in_weekend_nights', 'stays_in_week_nights', 'adu
lts', 'children', 'babies', 'is_repeated_guest', 'previous_cancellations', 'previ
ous_bookings_not_canceled', 'booking_changes', 'agent', 'company', 'days_in_waiti
ng_list', 'adr', 'required_car_parking_spaces', 'total_of_special_requests']
Number of numerical features: 20
```

In [10]:
```
# ============================================================
# STEP 10 — IDENTIFY CATEGORICAL FEATURES
# ============================================================

print("\n" + "=" * 70)
print("STEP 10 — CATEGORICAL FEATURES")
print("=" * 70)

categorical_columns = df.select_dtypes(
    exclude=np.number
).columns.tolist()

print("Categorical columns:")
```

```
print(categorical_columns)

print("Number of categorical features:", len(categorical_columns))
```

```
======================================================================
STEP 10 — CATEGORICAL FEATURES
======================================================================
Categorical columns:
['hotel', 'arrival_date_month', 'meal', 'country', 'market_segment', 'distributio
n_channel', 'reserved_room_type', 'assigned_room_type', 'deposit_type', 'customer
_type', 'reservation_status', 'reservation_status_date', 'city']
Number of categorical features: 13
```

In [11]:
```
# ============================================================
# STEP 11 — HANDLE CHILDREN MISSING VALUES
# ============================================================

print("\n" + "=" * 70)
print("STEP 11 — HANDLE CHILDREN MISSING VALUES")
print("=" * 70)

if "children" in df.columns:
    print("Missing children values before:", df["children"].isnull().sum())

    df["children"] = df["children"].fillna(0)

    print("Missing children values after:", df["children"].isnull().sum())

else:
    print("Children column not found.")
```

```
======================================================================
STEP 11 — HANDLE CHILDREN MISSING VALUES
======================================================================
Missing children values before: 4
Missing children values after: 0
```

In [12]:
```
# ============================================================
# STEP 12 — HANDLE COUNTRY MISSING VALUES
# ============================================================

print("\n" + "=" * 70)
print("STEP 12 — HANDLE COUNTRY MISSING VALUES")
print("=" * 70)

if "country" in df.columns:
    print("Missing country values before:", df["country"].isnull().sum())

    df["country"] = df["country"].fillna("Unknown")

    print("Missing country values after:", df["country"].isnull().sum())

else:
    print("Country column not found.")
```

```
======================================================================
STEP 12 — HANDLE COUNTRY MISSING VALUES
======================================================================
Missing country values before: 488
Missing country values after: 0
```

In [13]:
```
# ============================================================
# STEP 13 — HANDLE AGENT MISSING VALUES
# ============================================================

print("\n" + "=" * 70)
print("STEP 13 — HANDLE AGENT MISSING VALUES")
print("=" * 70)

if "agent" in df.columns:
    print("Missing agent values before:", df["agent"].isnull().sum())

    df["agent"] = df["agent"].fillna(0)

    print("Missing agent values after:", df["agent"].isnull().sum())

else:
    print("Agent column not found.")
```

```
======================================================================
STEP 13 — HANDLE AGENT MISSING VALUES
======================================================================
Missing agent values before: 16340
Missing agent values after: 0
```

In [14]:
```
# ============================================================
# STEP 14 — HANDLE COMPANY MISSING VALUES
# ============================================================

print("\n" + "=" * 70)
print("STEP 14 — HANDLE COMPANY MISSING VALUES")
print("=" * 70)

if "company" in df.columns:
    print("Missing company values before:", df["company"].isnull().sum())

    df["company"] = df["company"].fillna(0)

    print("Missing company values after:", df["company"].isnull().sum())

else:
    print("Company column not found.")
```

```
======================================================================
STEP 14 — HANDLE COMPANY MISSING VALUES
======================================================================
Missing company values before: 112593
Missing company values after: 0
```

In [15]:
```
# ============================================================
# STEP 15 — CREATE TOTAL NIGHTS FEATURE
# ============================================================

print("\n" + "=" * 70)
print("STEP 15 — CREATE TOTAL NIGHTS")
print("=" * 70)

if all(col in df.columns for col in [
    "stays_in_weekend_nights",
    "stays_in_week_nights"
]):
```

```
    df["total_nights"] = (
        df["stays_in_weekend_nights"]
        + df["stays_in_week_nights"]
    )

    print("Total nights feature created.")

    print(
        df[
            [
                "stays_in_weekend_nights",
                "stays_in_week_nights",
                "total_nights"
            ]
        ].head()
    )

else:
    print("Required night columns not found.")
```

```
======================================================================
STEP 15 — CREATE TOTAL NIGHTS
======================================================================
Total nights feature created.
   stays_in_weekend_nights  stays_in_week_nights  total_nights
0                        0                     0             0
1                        0                     0             0
2                        0                     1             1
3                        0                     1             1
4                        0                     2             2
```

In [16]:

```
# ============================================================
# STEP 16 — CREATE TOTAL GUESTS FEATURE
# ============================================================

print("\n" + "=" * 70)
print("STEP 16 — CREATE TOTAL GUESTS")
print("=" * 70)

guest_columns = [
    col for col in ["adults", "children", "babies"]
    if col in df.columns
]

if guest_columns:
    df["total_guests"] = (
        df[guest_columns].fillna(0).sum(axis=1)
    )

    print("Total guests feature created.")
    print(df["total_guests"].describe())

else:
    print("Guest columns not found.")
```

```
======================================================================
STEP 16 — CREATE TOTAL GUESTS
======================================================================
Total guests feature created.
count    119390.000000
mean          1.968239
std           0.722394
min           0.000000
25%           2.000000
50%           2.000000
75%           2.000000
max          55.000000
Name: total_guests, dtype: float64
```

In [17]:

```python
# ============================================================
# STEP 17 — CREATE ARRIVAL DATE
# ============================================================

print("\n" + "=" * 70)
print("STEP 17 — CREATE ARRIVAL DATE")
print("=" * 70)

month_map = {
    "January": 1,
    "February": 2,
    "March": 3,
    "April": 4,
    "May": 5,
    "June": 6,
    "July": 7,
    "August": 8,
    "September": 9,
    "October": 10,
    "November": 11,
    "December": 12
}

if "arrival_date_month" in df.columns:
    df["arrival_month_num"] = df[
        "arrival_date_month"
    ].map(month_map)

    if all(col in df.columns for col in [
        "arrival_date_year",
        "arrival_date_day_of_month"
    ]):
        df["arrival_date"] = pd.to_datetime(
            {
                "year": df["arrival_date_year"],
                "month": df["arrival_month_num"],
                "day": df["arrival_date_day_of_month"]
            },
            errors="coerce"
        )

        print("Arrival date feature created.")
        print(df[["arrival_date"]].head())

else:
    print("Arrival month column not found.")
```

```
======================================================================
STEP 17 — CREATE ARRIVAL DATE
======================================================================
Arrival date feature created.
  arrival_date
0   2024-07-27
1   2024-04-28
2   2024-09-10
3   2024-08-14
4   2024-09-14
```

In [18]:

```python
# ============================================================
# STEP 18 — CHECK INVALID GUEST RECORDS
# ============================================================

print("\n" + "=" * 70)
print("STEP 18 — INVALID GUEST RECORDS")
print("=" * 70)

if "total_guests" in df.columns:
    zero_guest_count = (df["total_guests"] == 0).sum()

    print("Bookings with zero guests:", zero_guest_count)

    print(
        "These records are reported for review, "
        "not automatically deleted."
    )
# ============================================================
# STEP 19 — OUTLIER ANALYSIS USING IQR
# ============================================================

print("\n" + "=" * 70)
print("STEP 19 — OUTLIER ANALYSIS")
print("=" * 70)

outlier_results = []

for column in df.select_dtypes(include=np.number).columns:

    values = df[column].dropna()

    if values.empty:
        continue

    q1 = values.quantile(0.25)
    q3 = values.quantile(0.75)
    iqr = q3 - q1

    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    outlier_count = (
        (values < lower_bound) |
        (values > upper_bound)
    ).sum()

    outlier_results.append({
        "Feature": column,
        "Q1": q1,
```

```
        "Q3": q3,
        "IQR": iqr,
        "Lower_Bound": lower_bound,
        "Upper_Bound": upper_bound,
        "Outlier_Count": int(outlier_count)
    })

outlier_df = pd.DataFrame(outlier_results)

print(outlier_df)

outlier_df.to_csv(
    os.path.join(
        PREPROCESSING_FOLDER,
        "step_19_outlier_analysis.csv"
    ),
    index=False
)

print("Outliers identified; not automatically removed.")
# ============================================================
# STEP 20 — SAVE PREPROCESSED DATASET
# ============================================================

print("\n" + "=" * 70)
print("STEP 20 — SAVE PREPROCESSED DATASET")
print("=" * 70)

CLEANED_FILE = os.path.join(
    PREPROCESSING_FOLDER,
    "hotel_bookings_cleaned.csv"
)

df.to_csv(CLEANED_FILE, index=False)

print("Cleaned dataset saved:", CLEANED_FILE)
print("Dataset shape:", df.shape)
# ============================================================
# STEP 21 — FINAL PREPROCESSING SUMMARY
# ============================================================

print("\n" + "=" * 70)
print("STEP 21 — PREPROCESSING SUMMARY")
print("=" * 70)

print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("Missing values remaining:", df.isnull().sum().sum())
print("Duplicate records remaining:", df.duplicated().sum())
print("Cleaned file:", CLEANED_FILE)

print("\nMEMBER 2 — PREPROCESSING COMPLETED")


# ============================================================
# STEP 22 — LOAD CLEANED DATASET
# MEMBER 3: EDA & STATISTICAL ANALYSIS
# ============================================================

print("\n" + "=" * 70)
```

```
print("STEP 22 — LOAD CLEANED DATASET")
print("=" * 70)

df = pd.read_csv(CLEANED_FILE)

print("Rows    :", df.shape[0])
print("Columns :", df.shape[1])
```

```
======================================================================
STEP 18 — INVALID GUEST RECORDS
======================================================================
Bookings with zero guests: 180
These records are reported for review, not automatically deleted.

======================================================================
STEP 19 — OUTLIER ANALYSIS
======================================================================
                           Feature       Q1      Q3     IQR  Lower_Bound  \
0                      is_canceled     0.00     1.0    1.00       -1.500
1                        lead_time    18.00   160.0  142.00     -195.000
2                arrival_date_year  2024.00  2024.0    0.00     2024.000
3         arrival_date_week_number    13.00    39.0   26.00      -26.000
4        arrival_date_day_of_month     8.00    23.0   15.00      -14.500
5          stays_in_weekend_nights     0.00     2.0    2.00       -3.000
6             stays_in_week_nights     1.00     3.0    2.00       -2.000
7                           adults     2.00     2.0    0.00        2.000
8                         children     0.00     0.0    0.00        0.000
9                           babies     0.00     0.0    0.00        0.000
10               is_repeated_guest     0.00     0.0    0.00        0.000
11          previous_cancellations     0.00     0.0    0.00        0.000
12  previous_bookings_not_canceled     0.00     0.0    0.00        0.000
13                booking_changes     0.00     0.0    0.00        0.000
14                           agent     7.00   152.0  145.00     -210.500
15                         company     0.00     0.0    0.00        0.000
16            days_in_waiting_list     0.00     0.0    0.00        0.000
17                             adr    69.29   126.0   56.71      -15.775
18     required_car_parking_spaces     0.00     0.0    0.00        0.000
19       total_of_special_requests     0.00     1.0    1.00       -1.500
20                    total_nights     2.00     4.0    2.00       -1.000
21                    total_guests     2.00     2.0    0.00        2.000
22                arrival_month_num     3.00     9.0    6.00       -6.000

    Upper_Bound  Outlier_Count
0         2.500              0
1       373.000           3005
2      2024.000              0
3        78.000              0
4        45.500              0
5         5.000            265
6         6.000           3354
7         2.000          29710
8         0.000           8590
9         0.000            917
10        0.000           3810
11        0.000           6484
12        0.000           3620
13        0.000          18076
14      369.500           1254
15        0.000           6797
16        0.000           3698
17      211.065           3793
18        0.000           7416
19        2.500           2877
20        7.000           5257
21        2.000          37339
22       18.000              0
Outliers identified; not automatically removed.
```

```
======================================================================
STEP 20 — SAVE PREPROCESSED DATASET
======================================================================
Cleaned dataset saved: output\preprocessing_results\hotel_bookings_cleaned.csv
Dataset shape: (119390, 37)

======================================================================
STEP 21 — PREPROCESSING SUMMARY
======================================================================
Rows: 119390
Columns: 37
Missing values remaining: 0
Duplicate records remaining: 0
Cleaned file: output\preprocessing_results\hotel_bookings_cleaned.csv

MEMBER 2 — PREPROCESSING COMPLETED

======================================================================
STEP 22 — LOAD CLEANED DATASET
======================================================================
Rows    : 119390
Columns : 37
```

In [19]:
```python
# ============================================================
# STEP 18 — CHECK INVALID GUEST RECORDS
# ============================================================

print("\n" + "=" * 70)
print("STEP 18 — INVALID GUEST RECORDS")
print("=" * 70)

if "total_guests" in df.columns:
    zero_guest_count = (df["total_guests"] == 0).sum()

    print("Bookings with zero guests:", zero_guest_count)

    print(
        "These records are reported for review, "
        "not automatically deleted."
    )
```

```
======================================================================
STEP 18 — INVALID GUEST RECORDS
======================================================================
Bookings with zero guests: 180
These records are reported for review, not automatically deleted.
```

In [20]:
```python
# ============================================================
# STEP 19 — OUTLIER ANALYSIS USING IQR
# ============================================================

print("\n" + "=" * 70)
print("STEP 19 — OUTLIER ANALYSIS")
print("=" * 70)

outlier_results = []

for column in df.select_dtypes(include=np.number).columns:

    values = df[column].dropna()
```

```
    if values.empty:
        continue

    q1 = values.quantile(0.25)
    q3 = values.quantile(0.75)
    iqr = q3 - q1

    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    outlier_count = (
        (values < lower_bound) |
        (values > upper_bound)
    ).sum()

    outlier_results.append({
        "Feature": column,
        "Q1": q1,
        "Q3": q3,
        "IQR": iqr,
        "Lower_Bound": lower_bound,
        "Upper_Bound": upper_bound,
        "Outlier_Count": int(outlier_count)
    })

outlier_df = pd.DataFrame(outlier_results)

print(outlier_df)

outlier_df.to_csv(
    os.path.join(
        PREPROCESSING_FOLDER,
        "step_19_outlier_analysis.csv"
    ),
    index=False
)

print("Outliers identified; not automatically removed.")
```

```
======================================================================
STEP 19 — OUTLIER ANALYSIS
======================================================================
                          Feature       Q1      Q3     IQR  Lower_Bound  \
0                      is_canceled     0.00     1.0    1.00       -1.500
1                        lead_time    18.00   160.0  142.00     -195.000
2                arrival_date_year  2024.00  2024.0    0.00     2024.000
3          arrival_date_week_number    13.00    39.0   26.00      -26.000
4         arrival_date_day_of_month     8.00    23.0   15.00      -14.500
5           stays_in_weekend_nights     0.00     2.0    2.00       -3.000
6              stays_in_week_nights     1.00     3.0    2.00       -2.000
7                            adults     2.00     2.0    0.00        2.000
8                          children     0.00     0.0    0.00        0.000
9                            babies     0.00     0.0    0.00        0.000
10               is_repeated_guest     0.00     0.0    0.00        0.000
11           previous_cancellations     0.00     0.0    0.00        0.000
12  previous_bookings_not_canceled     0.00     0.0    0.00        0.000
13                 booking_changes     0.00     0.0    0.00        0.000
14                           agent     7.00   152.0  145.00     -210.500
15                         company     0.00     0.0    0.00        0.000
16            days_in_waiting_list     0.00     0.0    0.00        0.000
17                             adr    69.29   126.0   56.71      -15.775
18     required_car_parking_spaces     0.00     0.0    0.00        0.000
19       total_of_special_requests     0.00     1.0    1.00       -1.500
20                    total_nights     2.00     4.0    2.00       -1.000
21                    total_guests     2.00     2.0    0.00        2.000
22               arrival_month_num     3.00     9.0    6.00       -6.000

    Upper_Bound  Outlier_Count
0         2.500              0
1       373.000           3005
2      2024.000              0
3        78.000              0
4        45.500              0
5         5.000            265
6         6.000           3354
7         2.000          29710
8         0.000           8590
9         0.000            917
10        0.000           3810
11        0.000           6484
12        0.000           3620
13        0.000          18076
14      369.500           1254
15        0.000           6797
16        0.000           3698
17      211.065           3793
18        0.000           7416
19        2.500           2877
20        7.000           5257
21        2.000          37339
22       18.000              0
Outliers identified; not automatically removed.
```

In [21]:
```
# ============================================================
# STEP 20 — SAVE PREPROCESSED DATASET
# ============================================================

print("\n" + "=" * 70)
print("STEP 20 — SAVE PREPROCESSED DATASET")
```

```
print("=" * 70)

CLEANED_FILE = os.path.join(
    PREPROCESSING_FOLDER,
    "hotel_bookings_cleaned.csv"
)

df.to_csv(CLEANED_FILE, index=False)

print("Cleaned dataset saved:", CLEANED_FILE)
print("Dataset shape:", df.shape)
```

```
======================================================================
STEP 20 — SAVE PREPROCESSED DATASET
======================================================================
Cleaned dataset saved: output\preprocessing_results\hotel_bookings_cleaned.csv
Dataset shape: (119390, 37)
```

In [22]:
```
# ============================================================
# STEP 21 — FINAL PREPROCESSING SUMMARY
# ============================================================

print("\n" + "=" * 70)
print("STEP 21 — PREPROCESSING SUMMARY")
print("=" * 70)

print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("Missing values remaining:", df.isnull().sum().sum())
print("Duplicate records remaining:", df.duplicated().sum())
print("Cleaned file:", CLEANED_FILE)

print("\nMEMBER 2 — PREPROCESSING COMPLETED")
```

```
======================================================================
STEP 21 — PREPROCESSING SUMMARY
======================================================================
Rows: 119390
Columns: 37
Missing values remaining: 0
Duplicate records remaining: 0
Cleaned file: output\preprocessing_results\hotel_bookings_cleaned.csv

MEMBER 2 — PREPROCESSING COMPLETED
```